Tensor Omics (TOX)

Analyze gene expression data and evolutionary relationships between genes

User's Guide

Aaron Schröder, Alexander Schwarzpaul, Anna Beketova, Asis Hallab, Franz-Eric Sill, Jitu Daba, Jonathan Kurz, Laszlo Lang, Luka Faensen, Mohamed Akdi, Stephanie Ped, Vivian Bass Vega

University of Applied Sciences Bingen, Germany

Last revised September 3, 2026

Contents

Part I.

Analysis Recipes

1. Detecting Divergent Genes: Outliers and Shift Vectors

Purpose

Find the genes whose expression has moved away from the rest of their gene family further than the family's own spread explains, and describe *how* each one moved. This is the original Tensor Omics workflow: the distances answer whether a copy diverged, the shift vectors answer in which direction.

Maintainer: Tensor Omics Developers

Required Input Data

- Normalized expression vectors, one gene per column, from the normalization sub-recipe.
- A gene-to-family mapping, with `0` for genes belonging to no family. Genes outside your families may stay in the matrix.
- Family centroids, from the centroid sub-recipe — and the decision made there about which members define them.

Prerequisites / Depends on

- *Normalization of Expression Values* (see chapter 4)
- *Gene Family Centroids* (see chapter 6)

Step-by-Step Workflow

Step 1. Measure each gene against its own family's centroid with `distance_to_centroid`. Genes with no family receive `-1`, a sentinel rather than a distance.

Step 2. Call `detect_outliers`. It does *not* compare distances to a fixed cutoff: it divides each distance by its family's own spread — the Relative Distance Index (RDI) — and flags the top $1 -$ `percentile` of those. Scaling per family is what puts a broadly spread family and a tight one on the same footing.

Step 3. Compute the shift vector field with `compute_shift_vector_field`. For each gene it returns two vectors, the family centroid and then the shift $\vec{s} = \vec{p} - \vec{o}$ from that centroid to the gene.

Step 4. Report the flagged genes with their shift vectors. Use the *raw* distances and shifts for everything downstream — the RDI exists to choose which genes are worth looking at, not to describe how far they moved.

Note

The family scaling is not simply each family's own standard deviation. A family with few members cannot give a usable one, so Tensor Omics fits the spread of a family against its mean distance across *all* families with LOESS, and reads each family's scale off that curve. A small family is therefore scaled by the trend its neighbours establish instead of by its own noise.

Example script

Runs the whole chain — centroids, distances, outliers, shift vectors — and writes a per-gene table plus the shift vectors of the flagged genes. Run unchanged from the repository root it takes about two seconds on the bundled data and writes `results/outliers.tsv`.

python/examples/outlier_detection.py

```python
from tensor_omics import (distance_to_centroid, detect_outliers,
                          compute_shift_vector_field)

# --- how far is each gene from its own family's centroid? --
distances = distance_to_centroid(
    expr, centroids, gene_to_family)   # -1 where no family

# --- which of those distances are extreme for their family? -
result = detect_outliers(
    n_families, distances, gene_to_family,
    percentile=0.95)                   # FRACTION: top 5%
is_outlier = result["is_outlier"]
quantile   = result["quantile"]        # effect size, NOT a p-value

# --- in which direction did each gene move? ----------------
field = compute_shift_vector_field(
    expr, centroids, gene_to_family)   # (n_axes, 2, n_genes)
centroid_of = field[:, 0, :]           # the family's centroid
shift       = field[:, 1, :]           # p - o, the divergence
```

Listing 1.1: Fragments to edit: the threshold, and what you keep

Important Parameters

Parameter	Controls	When / which values
`percentile`	Where the threshold on the RDI sits; the genes above it are flagged	A *fraction* in [0, 1], not a percentage: `0.95` keeps the top 5%. Lower it to screen more broadly, raise it to report only the strongest divergence
`n_families`	How many families the mapping refers to	Must cover the largest index in `gene_to_fam`, and must be the same number the centroids were computed with
`gene_to_fam`	Which family each gene is measured against	`0` for a gene belonging to no family. Such genes pass through the whole chain without being flagged and without affecting the threshold

Expected Results

With the default `percentile=0.95` the flagged fraction is essentially 5% of the genes that have a family — on the bundled data, 512 of 10 234, exactly 5.00%, with 75 594 genes belonging to no family carried along and none of them flagged. Distances to the own centroid have a median of

1.73 and a maximum of 2.96 there. A flagged fraction far from 1 – `percentile` means genes are reaching the test that should not: check for families with a zero centroid first.

⚠ Pitfall: reading percentile as a percentage

`percentile` is a fraction in [0, 1]. Passing `95` does not ask for the top 5% — it is out of range and the call fails. Passing `0.05` silently asks for the top 95%, flagging almost everything.

⚠ Pitfall: treating the quantile as a p-value

The `quantile` output states how extreme a gene's distance is *relative to the distances actually observed* — an empirical upper-tail effect size. It is not a null-hypothesis test: nothing here models what the distances would look like in the absence of divergence, so a `quantile` of 0.001 is not a p of 0.001 and must not be corrected for multiple testing as though it were.

⚠ Pitfall: a family whose spread cannot be estimated

A family with a single member *is* its own centroid: the distance is zero, there is no intra-family spread to estimate, its scaling factor is zero, and the gene can never be flagged. That is the right answer — one copy cannot diverge from itself — but it is silent, so a family reduced to one member by filtering leaves the analysis without a word. Check family sizes rather than trusting an empty result. And where too few families remain to fit the trend at all, every family falls back to a single global spread, losing exactly the between-family comparability the RDI exists to provide.

⚠ Pitfall: testing genes against a reference that contains them

If the centroids came from `group_centroid_all`, every candidate is part of the reference it is measured against, which pulls the centroid towards the divergent copies and shrinks exactly the distances this recipe looks for. Prefer orthologs-only centroids when the question is whether a copy diverged — see *Gene Family Centroids* (chapter 6).

⚠ Pitfall: averaging the distance sentinel

`distance_to_centroid` returns `-1` for a gene with no family. That is a marker, not a short distance: summarising the distance column without excluding it drags every statistic down. Select on the family mapping, not on the distances.

🔍 Interpretation

A flagged gene is one that sits further from its family's consensus than that family's own spread makes ordinary. It is a statement about *relative* divergence: the same absolute distance is unremarkable in a widely spread family and striking in a tight one, which is the whole point of scaling per family.
The distance says only *that* a gene moved. The shift vector says *where* to, and that is the interpretable part: its large components name the tissues that carry the divergence. A shift concentrated on one axis is a copy that gained or lost expression in that one tissue; a shift spread evenly across axes is a change in overall level rather than in pattern. Read the shift vectors of the flagged genes before drawing conclusions — and carry the raw shifts, not the RDI, into any later geometric analysis.
Finally, the flagged set is a *ranking cut*, not a discovery set with an error rate. At `percentile=0.95` you get the top 5% by construction, whether or not anything in your data has genuinely diverged.

References

- Cleveland (1979). Robust locally weighted regression and smoothing scatterplots. *Journal of the American Statistical Association* 74(368):829–836.

2. Subfunctionalization, Neofunctionalization and Dosage-Effect Detection (SND)

Purpose

Detect genes whose expression trajectories diverge between duplicated or orthologous copies in a way consistent with subfunctionalization, neofunctionalization, or dosage effects, using the Tensor Omics geometric framework.

Maintainer: Tensor Omics Developers

Required Input Data

- Per-sample expression matrices (e.g. TPM from `kallisto`), one matrix per species, genes × conditions.
- A sample sheet mapping each column to a condition and replicate.
- Gene-family assignments and ortholog/paralog relationships (produced by the prerequisites below).

Prerequisites / Depends on

- *Gene Family Detection* (see chapter 5)

Note

SND compares copies *within* a gene family, so the family grouping and the ortholog/paralog classification must be finished and sanity-checked before you start here.

Step-by-Step Workflow

Step 1. Normalize each species' expression matrix and assemble a TOX dataset (see the normalization and dataset-creation sub-recipes).

Step 2. Restrict the dataset to families that contain at least two copies in the compared lineages.

Step 3. Compute, per family, the geometric divergence of each copy's expression trajectory relative to the family centroid.

Step 4. Classify each divergent copy as subfunctionalized, neofunctionalized, or dosage-affected using the decision thresholds below.

Step 5. Export the per-gene SND table and the diagnostic trajectory plots.

Example script

Runs the full SND analysis on the bundled example data and writes an `snd_results.tsv` table plus per-family trajectory plots. Running it unchanged reproduces the default results referenced in this recipe.

`python/examples/snd_detection.py`

```
# --- input data
    ------------------------------------------------------------
species = {
    "ath": "data/ath_tpm.tsv",    # Arabidopsis thaliana
    "aly": "data/aly_tpm.tsv",    # Arabidopsis lyrata
}
sample_sheet = "data/samples.tsv"

# --- parameters (see the parameter table below)
    ----------------------------
min_family_size    = 2
divergence_cutoff = 0.35      # angular divergence from the family
    centroid
dosage_log2fc      = 1.0      # |log2 fold-change| flagged as a dosage
    effect
```

Listing 2.1: Fragments to edit: input paths and thresholds

Important Parameters

Parameter	Controls	When / which values
`min_family_size`	Smallest number of copies a family must have to be tested	Keep at 2 for pairwise comparisons; raise to focus on larger, higher-confidence families
`divergence_cutoff`	Angular divergence from the family centroid above which a copy is called "divergent"	Lower (~0.2) for sensitive screening; raise (~0.5) to report only strong divergence
`dosage_log2fc`	Absolute log2 fold-change flagged as a dosage effect	Use `1.0` (2×) as a default; tighten for deep, low-noise data

Expected Results

After step 3 you should have one divergence score per copy, most near zero. On the example data roughly 5–10% of copies exceed `divergence_cutoff`; a much larger fraction usually indicates a normalization or batch problem rather than genuine biology.

Common Pitfall

Comparing raw TPM across species without a common normalization inflates divergence scores. Always assemble the TOX dataset from consistently normalized matrices before computing divergence.

Pitfall: unbalanced replicates

Families whose copies are measured in different conditions produce artefactual trajectory divergence. Restrict to the shared condition set, or impute with care, before classification.

Interpretation

A copy with high divergence but a conserved expression *shape* relative to a sibling suggests *dosage*

change; divergent shape with retained breadth suggests *subfunctionalization*; a novel expression domain absent from all other copies suggests *neofunctionalization*. Read the per-family trajectory plots alongside the table before drawing conclusions.

References

- Robinson, McCarthy & Smyth (2010). edgeR: a Bioconductor package for differential expression analysis of digital gene expression data. *Bioinformatics* 26(1):139–140.
- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.

Part II.

Shared Sub-Recipes

3. Creating a TOX Dataset

Purpose

Read your expression table and gene families into the arrays Tensor Omics works on, check that they agree with each other, and store them together as a `.txdata` archive you can reopen later. Every other recipe starts from the arrays this one produces.

Maintainer: Tensor Omics Developers

Required Input Data

- A gene × sample table (CSV/TSV): one identifier column, then one column per replicate or sample.
- A gene family table in OrthoFinder's `Orthogroups.tsv` layout: one family per row, one column per species, gene ids comma-separated.
- Nothing else — this is the entry point.

Prerequisites / Depends on

- None. Later recipes depend on this one, not the other way round.

Note

A TOX dataset is not an object. It is a handful of plain arrays that agree by *position*: column *i* of `expression` belongs to the gene named by `gene_ids[i]`, and entry *i* of `gene_to_family` names that gene's family in `family_ids`. Nothing enforces this at run time, and nothing owns the arrays — keeping them consistent is yours to do, which is what the validation step below is for.

Step-by-Step Workflow

Step 1. Measure your files first. The readers *fill arrays you allocate*, so the number of genes, the number of samples, the number of families and the width of the longest identifier all have to be known before the first read. Count them from the files.

Step 2. Read the identifiers with `read_gene_ids_from_tsv_file`, then the matrix with `read_expression_vectors_tsv` into an `(n_samples, n_genes)` column-major array you allocated, then the families with `read_orthofinder_file`. A gene in no family gets the sentinel `0`.

Step 3. Check the arrays against each other before you rely on them: identifiers unique, family indices in range, expression non-negative. These are cheap and they catch the mistakes that are silent later.

Step 4. Store what you have with `save_tox_data`. Each array is passed together with the name it gets inside the archive; you may store any subset, so a dataset without centroids yet is perfectly valid.

Step 5. Reopen with `read_tox_data_into`, which returns every member. Members that were never written come back with zero extents.

Example script

Reads the bundled tables with the library's own readers, validates the relationships, writes an archive and reads it back to show the round trip is exact. Run unchanged from the repository root it takes about eight seconds over 88 327 genes and writes `results/example.txdata`.

python/examples/tox_dataset.py

```
import numpy as np
from tensor_omics import (read_gene_ids_from_tsv_file,
                          read_expression_vectors_tsv,
                          read_orthofinder_file, save_tox_data,
                          read_tox_data_into)

# --- sizes FIRST: the readers fill arrays you allocate -----
n_genes, n_samples, id_width  = 88327, 67, 14
n_families, family_id_width   = 15512, 9

gene_ids = read_gene_ids_from_tsv_file(
    "expression.tsv", id_width, n_genes,
    n_header_rows=1, gene_col=1)

# filled in place, so allocate it in its final form
expression = np.zeros((n_samples, n_genes),
                      dtype=np.float64, order="F")
read_expression_vectors_tsv(
    file_list=["expression.tsv"], gene_ids=gene_ids,
    expression_vectors=expression,
    n_header_rows=1, gene_col=1,
    value_cols=np.arange(2, n_samples + 2, dtype=np.int32),
    start_row=1)

families = read_orthofinder_file(
    "Orthogroups.tsv", gene_ids, family_id_width,
    n_families, n_genes)

# --- store: each array WITH the name it gets in the archive
save_tox_data("dataset.txdata",
    gene_ids=gene_ids,   gene_ids_file="gene_ids.bin",
    expression=expression, expression_file="expression.bin")

back = read_tox_data_into("dataset.txdata")
```

Listing 3.1: Fragments to edit: the files, their shape, and what you store

Important Parameters

Parameter	Controls	When / which values
`gene_ids_strlen`	Width of the fixed-size character slot each identifier is stored in	Must be at least the longest identifier; measure it, do not guess. Anything longer is truncated, which silently breaks the match between the expression table and the family table

Parameter	Controls	When / which values
`value_cols`	Which columns of the table hold values, 1-based	`2 ...n_samples+1` for the usual layout of one identifier column followed by the samples. Getting the count wrong leaves rows of the matrix untouched at zero
`n_header_rows`	Rows to skip before the data	1 for a single header line
`*_file`	The name each array is given inside the archive	Required whenever its array is passed, and vice versa. The names are yours to choose; the manifest records which is which

Expected Results

On the bundled data the readers report 88 327 genes × 67 samples and 15 512 families, of which 81 960 genes are assigned to a family and 6 367 carry the unassigned sentinel. All four checks pass, and reading the archive back reproduces every array exactly. `get_tox_data_dims` then reports `gene_id_len=14` and `family_id_len=9` — the widths actually needed. The file is about 47 MB for a 47.3 MB matrix: the archive is a container, not compression, so expect roughly the size of the raw data.

Pitfall: sizing the arrays by guess

Every extent has to be supplied before the read, and a wrong one fails quietly rather than loudly. An identifier width below the longest id truncates identifiers, so genes stop matching between the two tables and the family assignment comes back mostly empty; a `value_cols` shorter than the sample count leaves the remaining rows of the matrix at zero. Derive all of them from the files, as the example script does.

Pitfall: passing an array without its member name

`save_tox_data` takes each array together with the name it gets inside the archive, and needs both. Passing only one half fails with `ToxInputError: invalid input arguments`, naming the missing argument. Do not work around it by dropping the array — that is the data you meant to keep.

Pitfall: assuming the archive validates itself

Nothing checks the arrays against each other on the way in or out. An archive whose `gene_to_family` is a different length from its `gene_ids` stores and reloads without complaint, and the mismatch only surfaces as a wrong answer several recipes later. Validate before storing.

Pitfall: reading a file that is not a TOX archive

The reader reports what went wrong at the file level, not at the schema level: a missing file gives `ToxIOError: could not open file`, an ordinary zip gives `Could not extract file from archive`, and an archive whose manifest lists nothing gives `could not read array data`. None of these means your analysis is wrong — they mean the path is.

Interpretation

What you have at the end is the dataset every other recipe names in its required inputs. The expression matrix here is still *raw*: normalize it before any distance or angle is computed from it, and store the normalized matrix rather than recomputing it, so that every later result refers to one known state of the data.

Treat the archive as the unit of reproducibility. It records the arrays and their agreed positions, but not how they were produced — not the normalization settings, not which centroid convention was used, not the outlier threshold. Keep that alongside it, because a `.txdata` on its own cannot tell you which of the choices in the recipes that follow were made.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.

4. Normalization of Expression Values

Purpose

Turn a replicate-level expression table into the gene × tissue matrix that every Tensor Omics analysis uses as its expression vectors: gene-wise scale removed, replicates averaged per tissue, dynamic range compressed. This is the shared entry point of the TOX pipeline; the geometry of every later recipe is fixed by the choices made here.

Maintainer: Tensor Omics Developers

Required Input Data

- A table of non-negative expression measurements (e.g. TPM from `kallisto`), genes × replicates, one column per biological replicate.
- The replicate layout: how many replicates belong to each tissue, condition or time point. The replicates of one tissue must occupy *adjacent* columns.
- For stress-response designs only: which tissue is the control and which the condition, for each pair you want a fold change of.

Prerequisites / Depends on

- None — this is the first step of every Tensor Omics analysis.

Step-by-Step Workflow

Step 1. Arrange the measurements as an `(n_replicates, n_genes)` column-major array, so that *one gene is one column*. Group the replicates of each tissue into adjacent rows and record the group sizes in `reps_per_tissue`, e.g. `[4, 5, 5]` for three tissues measured with four, five and five replicates.

Step 2. Decide whether to quantile-normalize across replicates (`use_quantile`). It is off by default; switch it on when the replicate distributions differ enough that the difference is technical rather than biological.

Step 3. Run `normalization_pipeline`. It scales each gene by a LOESS-stabilized standard deviation, optionally quantile-normalizes, averages the replicates of each tissue, and finally applies $x \mapsto \log_2(x + 1)$.

Step 4. Check the result against the expectations below before using it: the shape, the value range, and the number of genes that reached the output unscaled.

Step 5. Stress-response designs only: call `calc_fchange` on the *log-transformed* matrix to turn control/condition pairs into log2 fold changes, so each such axis carries e.g. “drought in root over control root” rather than an absolute level.

Example script

Reads a replicate-level TSV, derives the tissue layout from its header, runs the pipeline and writes the normalized matrix. Run unchanged from the repository root, it normalizes the bundled `material/kallisto_sex_data_no_na.tsv` (88 327 genes, 67 replicates, 13 tissues) in a few seconds and writes `results/normalized_expression.tsv`.

python/examples/normalization.py

```python
import numpy as np
from tensor_omics import normalization_pipeline

# --- input: one gene per COLUMN, -------------------------
#     replicates of a tissue in adjacent ROWS
expr = np.asfortranarray(raw_counts)  # (n_replicates, n_genes)

# --- tissue layout: one entry per tissue, ----------------
#     summing to n_replicates
reps_per_tissue = np.array([4, 5, 5], dtype=np.int32)

# --- parameters (see the table below) --------------------
normalized = normalization_pipeline(
    expr, reps_per_tissue,
    span=0.7,             # LOESS span, mean-vs-SD fit
    degree=2,             # LOESS polynomial degree
    use_quantile=False,   # quantile normalization
)                         # -> (n_tissues, n_genes)
```

Listing 4.1: Fragments to edit: the input, the tissue layout, the parameters

Note

Tensor Omics stores one expression vector per *column* because Fortran is column-major. A matrix handed over as genes × replicates is not rejected — it is silently interpreted as replicates × genes, and every downstream distance and angle is then meaningless. Check `expr.shape` before you call.

Important Parameters

Parameter	Controls	When / which values
`reps_per_tissue`	How the replicate rows are grouped into tissues; entry *i* is the number of adjacent rows belonging to tissue *i*	Must sum to `n_replicates`, otherwise the call fails with `Array size mismatch`. Replicate counts may differ between tissues
`span`	Fraction of the genes entering each local fit of the mean–SD LOESS curve	Keep the default `0.7`. Lower it only if the mean–SD trend is genuinely non-monotonic and the fit visibly oversmooths; too low a span makes the curve follow noise
`degree`	Polynomial degree of the local LOESS fits	Keep the default `2`. Use `1` for a strongly linear trend or very few genes

Parameter	Controls	When / which values
`use_quantile`	Whether the replicate distributions are forced onto a common distribution after gene-wise scaling	Default `False`. Switch on when replicates differ in distribution for technical reasons; leave off when the between-replicate differences are the signal

Expected Results

The result has shape `(n_tissues, n_genes)` and is read-only — call `.copy()` if you need to modify it. For a non-negative input every value is ≥ 0, since $\log_2(x+1) \geq 0$ there. On the bundled example data the values span $[0, 4.06]$ without quantile normalization and $[0, 4.27]$ with it, and 2 627 of the 88 327 genes (3%) have zero variance across their replicates and therefore reach the output unscaled. A value range wildly different from this, or a much larger unscaled fraction, points at the input layout rather than at biology.

Pitfall: replicates that are not adjacent

`reps_per_tissue` describes *contiguous blocks* of rows, not a grouping by name. If the replicate columns of a tissue are scattered through the table, the pipeline still runs and silently averages the wrong columns together. Sort the columns tissue by tissue first; the example script refuses to continue when it detects a non-adjacent tissue.

Pitfall: genes the standard-deviation fit cannot use

The scaling factor comes from a LOESS curve fitted over all genes, so a gene with zero variance across its replicates carries no mean-versus-SD information. Such genes are left out of the fit *and reach the output unscaled* — they are not dropped and not flagged. Count them, as the example script does. If fewer than five genes have non-zero variance, the whole call fails with `ToxInputError: invalid input arguments`; this is what you get for an all-zero matrix, or for a toy example with too few genes.

Pitfall: expecting quantile normalization by default

Quantile normalization is an *option*, not a pipeline stage: the default is scale → average → $\log_2(x+1)$. Two analyses that differ only in `use_quantile` live in different geometries and must not be compared. Record the flag alongside the results.

Pitfall: fold changes computed on the wrong matrix

`calc_fchange` *subtracts* one axis from another. That is a log2 fold change only if you pass the log-transformed matrix — which is what `normalization_pipeline` returns. Passing raw tissue averages instead yields a plain difference of expression levels that merely looks like a fold change.

Warning

`root_mean_sq_normalization` divides by $\sqrt{\text{mean}(x^2)}$, which is not a standard deviation. It is kept for possible future use and must not be used for normalization; use `normalization_pipeline` or `normalize_by_std_dev`.

Interpretation

After normalization, a gene's coordinates state *where its expression sits across tissues*, not how many transcripts were counted. Gene-wise scaling removes each gene's own expression magnitude, so two genes differing a thousandfold in abundance can be neighbours if their tissue profiles agree; the $\log_2(x+1)$ step then compresses the remaining dynamic range so that no single highly expressed tissue dominates a distance. Distances and angles in this space therefore compare the *shape* of expression across tissues.

Because each choice made above defines a different space, results obtained under different settings answer different questions and generally disagree. Agreement between them is evidence of a robust effect; disagreement is usually informative rather than an error. Report the normalization settings with any result derived from this matrix.

References

- Cleveland (1979). Robust locally weighted regression and smoothing scatterplots. *Journal of the American Statistical Association* 74(368):829–836.
- Cleveland & Devlin (1988). Locally weighted regression: an approach to regression analysis by local fitting. *Journal of the American Statistical Association* 83(403):596–610.
- Bolstad, Irizarry, Åstrand & Speed (2003). A comparison of normalization methods for high density oligonucleotide array data based on variance and bias. *Bioinformatics* 19(2):185–193.

5. Gene Family Detection

Purpose

Group protein-coding genes across species into gene families so that downstream analyses can compare orthologous and paralogous copies. This is a shared preprocessing step reused by several analysis recipes.

Maintainer: Tensor Omics Developers

Required Input Data

- One protein FASTA per species (primary transcripts only).
- A species list / directory layout expected by OrthoFinder.

Step-by-Step Workflow

Step 1. Run OrthoFinder on the per-species protein FASTAs to obtain orthogroups.

Step 2. Optionally refine large orthogroups with MCL clustering to split fused families.

Step 3. Emit a gene-to-family table used by downstream recipes.

Example script

Wraps OrthoFinder + optional MCL refinement and writes `orthogroups.tsv`. A default run on the example proteomes reproduces the family table used elsewhere in this manual.

`scripts/gene_family_detection.sh`

Important Parameters

Parameter	Controls	When / which values
`inflation (-I)`	MCL granularity when refining orthogroups	Higher (`2.0`–`4.0`) yields smaller, tighter families; lower yields larger, more inclusive families

Common Pitfall

Including multiple splice isoforms per gene inflates family sizes and biases every downstream comparison. Reduce each proteome to primary transcripts first.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.

6. Gene Family Centroids

Purpose

Reduce each gene family to a single expression vector, its centroid: the reference that later recipes measure divergence against. Distances to the centroid, shift vectors from it, and the outlier test built on those all inherit whichever set of family members you average here.

Maintainer: Tensor Omics Developers

Required Input Data

- Normalized expression vectors, one gene per column (`(n_axes, n_genes)`), from the normalization sub-recipe.
- A gene-to-family mapping: for each gene, the index of its family, or `0` for a gene that belongs to none.
- Optional, and only for the orthologs-only centroid: which genes are the orthologs. This is *your* data — it comes from an orthology pipeline (OrthoFinder, synteny/Cactus, OMAmer), not from Tensor Omics.

Prerequisites / Depends on

- *Normalization of Expression Values* (see chapter 4)
- *Gene Family Detection* (see chapter 5)

Step-by-Step Workflow

Step 1. Number the families $1 \ldots n$ and build the `gene_to_family` vector, one entry per column of the expression matrix. Use `0` for every gene that belongs to no family — it is the sentinel the library expects, and such genes are skipped.

Step 2. Choose which members define the reference. `group_centroid_all` averages every gene of the family; `group_centroid_orthologs` averages only the genes marked in `ortholog_set`. See the interpretation below — this choice is the substance of the step, not a detail.

Step 3. Compute the centroids. The result is an `(n_axes, n_families)` matrix holding *one centroid per column*, in the same layout and orientation as the expression vectors that went in.

Step 4. Count, per family, how many genes actually contributed. A family with no contributor is not reported by the library — it silently receives a zero centroid, which is a valid point in the space and will quietly distort everything measured against it.

Example script

Builds the gene-to-family mapping from an OrthoFinder-style family table, computes the centroids and writes them, reporting assignment coverage and any family left without a contributing gene. Run unchanged from the repository root, it computes all-genes centroids for the 458 bundled families over `material/normalization.tsv` and writes `results/family_centroids.tsv`; pass `--orthologs` with a list of gene ids to take the orthologs-only centroid instead.

`python/examples/gene_family_centroids.py`

```
import numpy as np
from tensor_omics import (group_centroid_all,
                          group_centroid_orthologs)

# --- one gene per COLUMN, as normalization produced it -----
expr = np.asfortranarray(normalized)   # (n_axes, n_genes)

# --- family index per gene, 1-based; 0 = no family ---------
gene_to_family = np.array([1, 1, 2, 0, 2], dtype=np.int32)
n_families = 2

# --- (a) the family's actual mean, paralogs included -------
centroids = group_centroid_all(
    expr, gene_to_family, n_families)

# --- (b) a conserved reference: orthologs only -------------
ortholog_set = np.array([True, False, True, False, True])
centroids = group_centroid_orthologs(
    expr, gene_to_family, n_families, ortholog_set)
#   -> (n_axes, n_families), one centroid per column
```

Listing 6.1: Fragments to edit: the mapping, and which members define the centroid

Note

For a centroid over an arbitrary set of genes — not a whole family — call `mean_vector(expr, gene_indices)` directly. Its `gene_indices` are 1-based column numbers, not a boolean mask.

Important Parameters

Parameter	Controls	When / which values
`gene_to_family`	Which family each gene belongs to; entry *i* is the family index of column *i* of the expression matrix	Values must be `1`…`n_families`, or `0` for an unassigned gene. Any other value fails the call
`n_families`	How many centroids are produced, i.e. the number of columns of the result	Must be at least as large as the largest index in `gene_to_family`. Indices above it fail the call; families with no genes below it come back as zero centroids
`ortholog_set`	Which genes contribute when the orthologs-only centroid is taken	Required by `group_centroid_orthologs`, one boolean per gene. A family with no ortholog among its members gets a zero centroid

Expected Results

The result is `(n_axes, n_families)`, read-only, and each column is the element-wise mean of the contributing genes — so every centroid lies inside the range its family's members span on each axis. On the bundled data the script assigns 10 234 of 85 828 genes to 458 families (the rest belong to none), family sizes run from 6 to 143 genes, and no family is left empty. It also reports the 361

family members that have no row in the expression table; a much larger number there means the gene identifiers of the two files do not match.

⚠ Pitfall: a family with no members gets a zero centroid

A family that no gene contributes to — because it has no members in the expression table, or none of them is an ortholog — does not raise an error and is not flagged. It receives the zero vector, which is an ordinary point at the origin of the space. Every later distance to that centroid is then simply the gene's own norm, so the family looks maximally divergent for a purely bookkeeping reason. Count the contributors per family and check the count is never zero before going on.

⚠ Pitfall: which members you average changes every later result

The two entry points are not variants of one number. On the bundled data the orthologs-only centroids shipped in `material/centroids_orthologs_only.tsv` sit a median 13.5% of their own length away from the all-genes centroids of the same families, and up to 1.56 units away — both computed from the same expression table, so the whole difference comes from the member set. Averaging *all* genes folds the very paralogs you may later test for divergence into the reference they are tested against, which pulls the reference toward them and hides them. Use the orthologs-only centroid whenever the analysis asks whether a copy has diverged.

⚠ Pitfall: marking unassigned genes with anything but zero

`0` is the sentinel for "no family", and genes carrying it are skipped. A negative index, or an index above `n_families`, is not treated as "unassigned" but fails the call with `ToxInputError: invalid input arguments (argument 'gene_to_family')`. Map every gene you want excluded to `0` explicitly rather than leaving a placeholder behind.

🔍 Interpretation

A centroid is the family's consensus expression profile, and the distance of a member from it is that member's divergence from the family. What the consensus *means* is decided by the member set. The orthologs-only centroid is a *conserved* reference: it describes where the family's expression sat before the copies you are testing diverged, which is what makes a large distance interpretable as divergence of that copy. The all-genes centroid is the family's actual centre of mass, including every divergent copy — the right reference when you want to describe the family as it is, and the wrong one when you want to detect a member that moved.

Read a centroid alongside its contributor count. A centroid over two genes is a midpoint, not a consensus, and distances measured against it carry the noise of those two genes.

References

- Emms & Kelly (2019). OrthoFinder: phylogenetic orthology inference for comparative genomics. *Genome Biology* 20:238.